

Tute (Week 10)

Church Numerals

This question will use the Church encoding of the natural numbers, as seen in the lectures. Here's a quick refresher:

$$\begin{aligned} 0 &= \lambda f x. x \\ 1 &= \lambda f x. f x \\ 2 &= \lambda f x. f(f x) \\ &\dots \end{aligned}$$

$$(+)=\lambda n m f x. n f (m f x)$$

1. You might expect that (+) is commutative, in the sense that (+) a b and (+) b a reduce to the same value for all a and b. That is not the case. Can you think of a counterexample? *Hint*: try feeding it things that aren't Church numerals.
2. How would you define multiplication of Church numerals in the lambda calculus?
3. How would you define exponentiation?
4. How would you define the predecessor function on Church numerals? In day-to-day mathematics this function could be defined as

$$\begin{aligned} \text{pre}(0) &= 0 \\ \text{pre}(n) &= n - 1 \quad \text{if } n > 0 \end{aligned}$$

Where the first clause is to prevent us from falling outside the natural numbers. This is sometimes called saturating subtraction.

This question is harder than the rest. You may want to save it for last.

5. Given a predecessor function as defined in the previous question, can you define saturating subtraction (where $n - m = 0$ if $m > n$)?
6. Can you define a function which, given two numbers, returns true iff they are equal?

Recall the Church encoding of the booleans:

$$\begin{aligned} \text{true} &= \lambda x y. x \\ \text{false} &= \lambda x y. y \end{aligned}$$

1. We have

$$(+) (\lambda f x. f) (\lambda f x. x x) \mapsto_{\beta}^* \lambda f x. f$$

but if we commute the arguments we get

$$(+) (\lambda f x. x x) (\lambda f x. f) \mapsto_{\beta}^* \lambda f x. f f$$

2. $(*) = \lambda n m. n ((+) m) 0$

3. $\text{exp} = \lambda n m. m ((* n) 1)$

4. $\text{pre} = \lambda n f x. (n (\lambda m g. g (m f)) (\lambda g. x)) (\lambda x. x)$

What's going on here? The intention is that the innermost expression

$n (\lambda m g. g (m f)) (\lambda g. x)$ is a copy of the numeral $n f x$, but with the outermost f abstracted. This will evaluate to an expression of the form $\lambda g. f^{n-1} x$. Instantiating g to be the identity function yields the predecessor of n , as desired.

5. $(-) = \lambda n m. m \text{ pre } n$

6. $\lambda n m. ((-) n m) (\lambda x. \text{false}) (((-) m n) (\lambda x. \text{false}) \text{true})$

The idea is that n equals m holds iff $n - m = 0$ and $m - n = 0$.

Fixpoint combinators

We briefly saw Turing's fixpoint combinator in the lectures, a lambda-calculus term which we will denote μ . Its exact definition is not important right now: what matters is that it satisfies

$$\mu f \mapsto_{\beta}^* f(\mu f)$$

1. Use μ to define a function which computes the n :th triangular number.

2. Use μ to define a function which computes the n :th Fibonacci number.

Hint: your function should take the form μf for some f that you'll have to write. Your f will be a function that takes two arguments: the first argument represents a continuation (intuitively, a function that you can call to initiate a recursive call) and the second argument will be the number n . You may find it helpful to use the auxiliary functions from the previous question.

1. $\mu (\lambda c n. (\text{eq } n 0) 0 ((+) n (c (\text{pre } n))))$

This is reminiscent of the following recursive function definition:

$$c(n) =$$

if $n = 0$ **then**
0
else
 $n + c(n - 1)$

2. $\mu (\lambda c n. (eq\ n\ 0)\ 0\ ((eq\ n\ 1)\ 1\ ((+)\ (c\ (pre\ (pre\ n)))\ (c\ (pre\ n)))))$

This is reminiscent of the following recursive function definition:

$$c(n) =$$

if $n = 0$ **then**
0
else if $n = 1$ **then**
1
else
 $c(n - 2) + c(n - 1)$